

Resum de Javadoc (Tomeu Estrany)

A NetBeans, l'ús de Javadoc és molt similar a com es fa servir en qualsevol altre entorn de desenvolupament de Java. Javadoc és una eina que genera documentació a partir de comentaris en el codi font de Java. Aquests comentaris estan escrits en un format especial i poden incloure etiquetes que Javadoc interpreta per generar documentació en format HTML.

Per utilitzar Javadoc a NetBeans, segueix aquests passos:

1. Escriu comentaris Javadoc: Dins del codi font de la teva classe Java, escriu comentaris Javadoc abans de les declaracions de classe, mètodes o variables que vulguis documentar. Els comentaris Javadoc comencen amb `/**` i acaben amb `*/`.

```
/**
 * Aquesta classe representa un exemple de classe.
 */
public class Exemple {

    /**
     * Aquest mètode realitza una acció específica.
     * @param parametro Descripció del paràmetre
     * @return Descripció del valor de retorn
     */
    public int metodeExemple(int parametro) {
        // Codi del mètode
    }
}
```

2. Genera la documentació Javadoc: A NetBeans, pots generar la documentació Javadoc seleccionant el teu projecte a l'Explorador de projectes i fent clic amb el botó dret. Llavors, selecciona "Generate Javadoc" al menú contextual. Això iniciarà el procés de generació de documentació Javadoc per al teu projecte.
3. Configura les opcions de generació: NetBeans et permetrà configurar diferents opcions per a la generació de la documentació, com la ubicació de sortida, els fitxers a incloure o excloure, i altres opcions avançades.
4. Revisa la documentació generada: Un cop NetBeans hagi completat el procés de generació, podràs trobar la documentació Javadoc generada a la ubicació especificada durant la configuració. La documentació estarà en format HTML i podràs obrir-la al teu navegador web per revisar-la.
5. Actualitza la documentació segons sigui necessari: És important mantenir actualitzada la documentació Javadoc a mesura que el codi canvia. Assegura't de

mantenir els teus comentaris Javadoc precisos i descriptius perquè la documentació sigui útil per a altres desenvolupadors que treballin amb el teu codi.

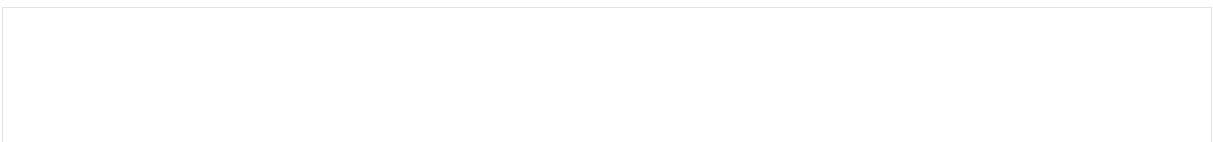
Amb aquests passos, podràs utilitzar Javadoc a NetBeans per generar documentació clara i completa per al teu codi Java!

Resum de algunes de les etiquetes Javadoc més rellevants:

1. **@param**: Descriu un paràmetre d'un mètode. S'utilitza per especificar el nom del paràmetre i proporcionar una descripció del mateix.
2. **@return**: Descriu el valor de retorn d'un mètode. S'utilitza per proporcionar una descripció del valor que retorna un mètode.
3. **@throws**: Indica les excepcions que pot llençar un mètode. S'utilitza per documentar les excepcions que poden ocórrer durant l'execució del mètode.
4. **@see**: Proporciona una referència a una altra classe, mètode o camp relacionat. S'utilitza per enllaçar la documentació d'un element amb un altre.
5. **@since**: Indica la versió en la qual es va introduir un element. S'utilitza per especificar la versió de l'API Java en la qual es va introduir una classe, mètode o camp.
6. **@deprecated**: Marca un element com a obsolet. S'utilitza per indicar que un element ja no es recomana utilitzar i proporcionar una alternativa si està disponible.
7. **@author**: Indica l'autor del codi. S'utilitza per especificar qui va escriure el codi font d'una classe, mètode o camp.
8. **@version**: Indica la versió d'una classe. S'utilitza per especificar la versió actual d'una classe.
9. **@inheritDoc**: Copia la documentació d'un mètode o camp des de la seva superclasse o interfície. S'utilitza per heretar la documentació d'un mètode o camp de la seva implementació base.
10. **@link**: Crea un enllaç a una altra classe, mètode o camp. S'utilitza per inserir enllaços a la documentació Javadoc.

Aquestes són només algunes de les etiquetes Javadoc més rellevants i útils per documentar codi Java de manera efectiva.

Aquí tens un exemple de com utilitzar les etiquetes Javadoc en un codi Java:



```

/**
 * Aquesta classe representa una calculadora bàsica.
 * Permet realitzar operacions de suma, resta, multiplicació i divisió.
 * @author Tomeu Estrany
 * @version 1.0
 */
public class Calculadora {

    /**
     * Realitza la suma de dos números.
     * @param a El primer número a sumar.
     * @param b El segon número a sumar.
     * @return La suma dels dos números.
     */
    public int sumar(int a, int b) {
        return a + b;
    }

    /**
     * Realitza la resta de dos números.
     * @param a El número del qual es resta.
     * @param b El número que es resta.
     * @return La resta dels dos números.
     * @throws IllegalArgumentException Si el resultat és negatiu.
     */
    public int restar(int a, int b) {
        if (a < b) {
            throw new IllegalArgumentException("El resultat és negatiu");
        }
        return a - b;
    }

    /**
     * Realitza la multiplicació de dos números.
     * @param a El primer número a multiplicar.
     * @param b El segon número a multiplicar.
     * @return El producte dels dos números.
     */
    public int multiplicar(int a, int b) {
        return a * b;
    }

    /**
     * Realitza la divisió de dos números.
     * @param a El dividend.
     * @param b El divisor.
     * @return El quocient de la divisió.
     * @throws ArithmeticException Si el divisor és 0.
     */
    public int dividir(int a, int b) {
        if (b == 0) {
            throw new ArithmeticException("No es pot dividir per zero");
        }
        return a / b;
    }
}

```

- S'utilitza **@param** per descriure els paràmetres dels mètodes.
- **@return** s'usa per descriure el valor de retorn dels mètodes.
- **@throws** s'utilitza per documentar les excepcions que poden ser llançades pels mètodes.
- **@author** especifica l'autor del codi.
- **@version** indica la versió de la classe.
- **@deprecated** es podria usar per marcar mètodes que han estat reemplaçats per altres.
- **@since** podria ser útil per indicar des de quina versió s'ha inclòs un mètode a la classe.

Un exemple amb més elements...

```

/**
 * Aquesta classe representa un servei de gestió d'usuaris.
 * Proporciona mètodes per crear, actualitzar i eliminar usuaris en un sistema.
 * @author Tomeu Estrany
 * @version 2.0
 */
public class GestionUsuarios {

    /**
     * Crea un nou usuari en el sistema.
     * @param nombre El nom de l'usuari.
     * @param edad L'edat de l'usuari.
     * @throws IllegalArgumentException Si el nom és nul o l'edat és menor que 0.
     * @return L'ID del nou usuari creat.
     */
    public int crearUsuario(String nombre, int edad) {
        if (nombre == null || edad < 0) {
            throw new IllegalArgumentException("Nom invàlid o edat negativa");
        }
        return 123; // ID del nou usuari
    }

    /**
     * Actualitza la informació d'un usuari existent en el sistema.
     * @param id L'ID de l'usuari a actualitzar.
     * @param nombre El nou nom de l'usuari.
     * @param edad La nova edat de l'usuari.
     * @throws IllegalArgumentException Si l'ID és negatiu, el nom és nul o l'edat és
     menor que 0.
     * @return true si l'actualització va ser exitosa, false si no es va trobar
     l'usuari amb l'ID especificat.
     */
    public boolean actualizarUsuario(int id, String nombre, int edad) {
        if (id < 0 || nombre == null || edad < 0) {
            throw new IllegalArgumentException("ID invàlid, nom nul o edat
negativa");
        }
        return true; // L'actualització va ser exitosa
    }

    /**
     * Elimina un usuari del sistema.
     * @param id L'ID de l'usuari a eliminar.
     * @throws IllegalArgumentException Si l'ID és negatiu.
     * @return true si l'usuari es va eliminar amb èxit, false si no es va trobar
     l'usuari amb l'ID especificat.
     */
    public boolean eliminarUsuario(int id) {
        if (id < 0) {
            throw new IllegalArgumentException("ID invàlid");
        }
        return true; // L'usuari es va eliminar amb èxit
    }

    /**
     * Obté la versió actual del servei de gestió d'usuaris.
     * @return La versió actual del servei.
     */
    public String getVersion() {
        return "2.0";
    }
}

```

```

/**
 * Mètode d'exemple per mostrar l'ús de l'etiqueta @deprecated.
 * Aquest mètode ha estat reemplaçat per un altre més eficient i hauria de
deixar-se d'utilitzar.
 * @deprecated Aquest mètode està obsolet i pot ser eliminat en futures versions.
 */
@Deprecated
public void metodoObsoleto() {
    // Codi del mètode obsolet
}

/**
 * Mètode d'exemple per mostrar l'ús de l'etiqueta @since.
 * Aquest mètode es va introduir a la versió 2.0 del servei de gestió d'usuaris.
 * @since 2.0
 */
public void metodoDesdeVersion2() {
    // Codi del mètode
}

/**
 * Mètode d'exemple que mostra l'ús de l'etiqueta @link.
 * Aquest mètode crea un enllaç a la documentació de
 *{@link #actualizarUsuario(int, String, int)}.
 * @see #actualizarUsuario(int, String, int)
 */
public void metodoConLink() {
    // Codi del mètode
}

/**
 * Mètode d'exemple que mostra l'ús de l'etiqueta @inheritDoc.
 * Aquest mètode sobreesciu el mètode {@link #toString()} de la classe Object.
 * @inheritDoc
 */
@Override
public String toString() {
    return super.toString();
}
}

```

- **@param:** S'utilitza en els mètodes crearUsuario, actualizarUsuario i eliminarUsuario per descriure els paràmetres de cada mètode.
- **@return:** S'emplea en els mètodes crearUsuario, actualizarUsuario, eliminarUsuario i getVersion per descriure el valor de retorn de cada mètode.
- **@throws:** S'utilitza en els mètodes crearUsuario, actualizarUsuario i eliminarUsuario per indicar les excepcions que poden ser llançades per aquests mètodes.
- **@since:** S'emplea en el mètode metodoDesdeVersion2 per indicar la versió en la qual es va introduir aquest mètode.
- **@deprecated:** S'utilitza en el mètode metodoObsoleto per marcar-lo com a obsolet i proporcionar una advertència sobre el seu ús.
- **@inheritDoc:** S'emplea en el mètode toString per heretar la documentació del mètode toString de la classe Object.

- **@author:** S'utilitza en la classe per especificar l'autor del codi.
- **@version:** S'usa en la classe per especificar la versió actual de la mateixa.
- **@link:** Aquesta etiqueta s'utilitza per crear un enllaç a una altra classe, mètode o camp relacionat dins de la documentació Javadoc. Per exemple, en el mètode `metodoConLink` de la classe `GestionUsuarios`, s'està creant un enllaç a la documentació del mètode `actualizarUsuario` utilitzant l'etiqueta `@link`.
- **@see:** Aquesta etiqueta també s'utilitza per crear enllaços, però és més versàtil. Permet enllaçar a una àmplia gamma de recursos, incloent altres classes, mètodes, camps, paquets i fins i tot documents externs. En l'exemple proporcionat, s'utilitza en el mètode `metodoConLink` per crear un enllaç a la documentació del mètode `actualizarUsuario`.

L'etiqueta `@see` en Javadoc és molt útil per referenciar altres recursos relacionats, com ara altres classes, mètodes, camps, paquets o fins i tot documents externs. Aquí tens algunes variants de com es pot utilitzar l'etiqueta `@see` amb exemples:

1. Referència a una altra classe:

```
/**
 * Classe que conté mètodes relacionats amb la gestió d'usuaris.
 * @see GestionUsuarios
 */
public class UserManagement {
    // Codi de la classe
}
```

En aquest exemple, l'etiqueta `@see` fa referència a la classe `GestionUsuarios`, indicant als desenvolupadors que hi ha més informació sobre la gestió d'usuaris en aquesta classe.

2. Referència a un mètode d'una altra classe:

```
/**
 * Classe que conté mètodes relacionats amb la gestió d'usuaris.
 * @see GestionUsuarios#crearUsuario(java.lang.String, int)
 */
public class UserManagement {
    // Codi de la classe
}
```

Aquí, l'etiqueta `@see` fa referència específicament al mètode `crearUsuario` de la classe `GestionUsuarios`, permetent als desenvolupadors accedir directament a la documentació d'aquest mètode.

3. Referència a un camp d'una altra classe:

```
/**
 * Classe que conté mètodes relacionats amb la gestió d'usuaris.
 * @see GestionUsuarios#VERSION
 */
public class UserManagement {
    // Codi de la classe
}
```

En aquest cas, **VERSION** pot ser un camp estàtic de la classe **GestionUsuarios**, i l'etiqueta **@see** permet als desenvolupadors accedir a la seva documentació.

4. Referència a un paquet:

```
/**
 * Classe que conté mètodes relacionats amb la gestió d'usuaris.
 * @see com.example.utilities
 */
public class UserManagement {
    // Codi de la classe
}
```

Aquí, l'etiqueta **@see** fa referència al paquet **com.example.utilities**, indicant als desenvolupadors que poden trobar recursos relacionats amb la gestió d'usuaris dins d'aquest paquet.

5. Referència a un document extern:

```
/**
 * Classe que conté mètodes relacionats amb la gestió d'usuaris.
 * @see <a href="https://www.example.com/documentacio">Documentació completa</a>
 */
public class UserManagement {
    // Codi de la classe
}
```

En aquest exemple, l'etiqueta **@see** enllaça amb un document extern a través d'una URL, proporcionant als desenvolupadors accés a una documentació més completa fora del codi font.